MANUAL TEST GUIDE · MODULE 04 OF 11

Booking Functional-Flow Test Walkthroughs

Scenario-driven, hands-on QA guide for **logistics.booking** — the conditional provider-reservation layer between Handover and Shipments. Centred on the **handover Y-fork** (in-house vs carrier-sourced), the 6-state booking machine, provider confirmation, feasibility, amendment, and cancellation. Every step traced to its BRS requirement ID.

> **How to use:** Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives you a precondition, an action, the expected result, and a ✔ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a guard to confirm it holds.

PREPARED FOR	SOURCE INPUTS	DEMO LANE	DATE
Manual QA / UAT	Booking BRS (QAB-12/14/17) · logistics/docs/booking.md · demo use-case	Globex INMUM → SGSIN	15 July 2026

SECTION 01

01 What Booking does & how to read this guide

A **booking** is the short-lived, conditional provider-reservation record that sits between the sales-to-ops **handover** and the long-lived **shipment**. It exists **only** when equipment must be sourced from a carrier. One model, `logistics.booking`; `transport_mode_id` is a row field, never a per-mode subclass. Charges are frozen at confirmation; amendments are the only legal mutation path past that (`BR-BK-04`).

17	3	4	3
COMMANDS TO EXERCISE	TEST SCENARIOS	STATE MACHINES	APPROVAL FLOWS

The signature flow — the handover Y-fork

read this first

At an **accepted** handover, operations pick *one* of two mutually-exclusive paths. The neutral routing slot `crm.handover.fulfillment_target_id` is owned by sales-crm and frozen once set (`BR-HO-ROUTE-01`) — a booking can never be created from an already-routed handover.

```
accepted crm.handover ─┬─ Path A · in-house equipment ──► Create Shipment ──► logistics.shipment  (booking skipped)
                       └─ Path B · carrier-sourced ──────► Create Booking ──► logistics.booking ──► shipment
```

The four state machines you will observe

```
booking     : draft → requested → awaiting_confirmation → confirmed → ready_for_execution | cancelled
provider.rq : draft → requested → awaiting → confirmed | rejected | amended | cancelled
feasibility : pending → feasible | exception | not_feasible
amendment   : draft → pending_approval / pending_customer_ack → applied | rejected | withdrawn
```

PROVIDER_KIND — 8 PARALLEL RESERVATIONS (QAB-14)

- `shipping_line` · `nvocc` · `airline` · `transporter`
- `cha` · `warehouse` · `overseas_agent` · `insurance`
- Immutable after create (`BR-BK-RQ-01`) — switch = reject + new row

FEASIBILITY 7 DIMS / AMENDMENT 6 CHANGE_TYPES

- **dims:** schedule · equipment · cargo · cut_off · route · vendor · compliance
- **change_type:** cargo_qty · schedule · carrier · route · equipment · charges
- `transport_mode` / `source_handover` are **not** amendable (`BR-BK-AMD-06`)

The scenarios in this guide

Scenario	What it proves	Key commands	Section
A — Y-fork Path B	Handover → booking carry-forward, provider request + confirm, readiness gate, and the Y-fork guards (already-routed freeze, handover-not-accepted)	`create_from_handover` , `provider.request.confirm` , `mark_ready`	§02 · §03
B — Feasibility + Amendment	7-dimension feasibility commit + escalation; amendment submit / withdraw / apply with commercial-review approval	`feasibility.check.commit` , `amendment.submit`	§04
C — Cancellation (guards)	grace-window vs cancel-fee approval, equipment allocate / release, and the negative guards that must refuse	`cancel` , `complete_cancellation`	§05

Demo fixtures you start from (loaded by `--with-demo=all`)

Forwarder **Acme Forwarding — Mumbai**; customer **Globex Inc.**; ocean carrier **Maersk Demo Line**. The seeded booking `demo_booking_globex_001` (2×40HC cotton garments, 18 t / 128 CBM, buy 3200 / sell 4250 USD) lands in **draft** with a pending feasibility row auto-created, plus 2 required provider requests (`shipping_line` Maersk, `transporter` DHL). Parties are intentionally omitted from the seed — add them on the form before `mark_ready` .

SECTION 02 · SCENARIO A — THE Y-FORK DEEP-DIVE

02 Path B — handover → booking → ready for execution

Goal: take an accepted Globex handover down the **carrier-sourced** fork to a booking that confirms its providers and reaches **ready_for_execution** with no re-keying. **Role:** Operations coordinator. **Start state:** a `crm.handover` in `accepted`, not yet routed.

#	Action	Expected result	✔ Checkpoint	Trace
1	On the accepted handover, click **Create Booking** (`create_from_handover`).	New booking in **draft**. Parties, route, cargo lines, schedule and **frozen charges** are copied from the handover + accepted quote version; `transport_mode_id` is derived from the service; `booking_ref` auto-stamped `BK-YYYY-NNNNNN` . The handover is routed (`fulfillment_target_id=booking`) and its `booking_id` back-set.	**Booking opens; every field carried, nothing re-typed.**	BR-09 BR-11
2	Observe the auto-created **feasibility** row on the booking (side-effect of `booking.created`).	One `logistics.booking.feasibility.check` exists in **pending**. Resolve it to **feasible** before requesting providers (see §04).	**Feasibility row present, result=pending.**	QAB-12 BR-BK-FEAS-01
3	Run **Request Provider** (`request_provider`).	Booking flips **draft → requested**, `requested_at` stamped. Gated on feasibility ∈ {feasible, exception} (`BR-BK-03`).	**status=requested.**	QAB-12 BR-BK-03
4	On the Maersk `shipping_line` request: `send` (draft→requested), then `confirm` with `confirmation_ref` + `vessel_name` , `voyage_number` , `etd` , `eta` , `si_cut_off` , `vgm_cut_off` .	Request → **confirmed**. Strictest cut-offs + primary-provider (vessel/voyage) **cascade onto the booking header**; `booking.provider.confirmed` event fires (`BR-BK-CONF-03`).	**Header shows Maersk + vessel; cut-offs populated.**	QAB-14 BR-BK-CONF-02
5	Confirm the DHL `transporter` request (`vehicle_reference` , `etd` , `eta` , `gate_in_cut_off`).	Both required requests now **confirmed** — the readiness gate's precondition 3 is satisfied.	**All `is_required` requests confirmed.**	QAB-14
6	Advance the booking to **confirmed** via the status bar (`status` is `workflow=True`).	Header walks **requested → awaiting_confirmation → confirmed**. There is *no* dedicated confirm command — this is a workflow-driven transition (statusbar / kanban drag).	**status=confirmed.**	BR-13 workflow
7	Run **Mark Ready** (`mark_ready`).	The precondition gate runs (see §03). All pass → **ready_for_execution**, `ready_at` stamped, equipment auto-allocated, and `Booking.ReadyForExecution` fires — **handing off to Shipments**.	**status=ready_for_execution; event fired.**	QAB-17 BR-BK-08

✔ Scenario A pass criteria

The booking reached **ready_for_execution** entirely from carried-forward handover data plus confirmed providers — and the `Booking.ReadyForExecution` event is the boundary that Shipments consumes to spawn the shipment.

⚠ Path A contrast — booking is skipped entirely

When equipment is **in-house** (from `logistics.equipment-operations`), ops instead clicks **Create Shipment** on the handover — no `logistics.booking` row is ever created. Feasibility (QAB-12) still runs, but on the handover-to-shipment transition, not here. Confirm that a Path-A handover has an empty `booking_id` .

SECTION 03 · SCENARIO A

03 The readiness gate & the Y-fork guards

Half of testing a commitment layer is confirming it **refuses** bad input. First the `mark_ready` precondition set; then the negative checks that each trip a specific guard with a real message.

The `mark_ready` precondition set (QAB-17 / BR-BK-08)

All failures accumulate in one pass — the command **never short-circuits**. On any failure it raises `BookingNotReady: N precondition(s) failed: [...]` and the status stays put; fix all listed items and re-run.

#	Condition checked	Failure reason surfaced if missing
1	Status is `confirmed`	"booking status must be `confirmed` to mark ready; got …"
2	Feasibility ∈ {feasible, exception}	"no feasibility check linked" / "feasibility result … blocks mark_ready"
3	Every `is_required` provider request confirmed	"provider requests not confirmed: […]" / "no required provider request rows"
4	Per-mode cut-offs present (sea: SI/VGM/gate-in)	"<cut_off> required for mode 'sea' but is null"
5	Required fields: `pol_id` , `pod_id` , `incoterm_id` , `currency_id` ; ≥1 line	"booking is missing required field: POL/POD/Incoterm/Currency"
6	Parties: shipper, consignee, notify present	"booking is missing party with role `shipper`/`consignee`/`notify`"
7	`operational_owner_id` assigned	"operational_owner_id must be assigned"
8	No pending amendments	"amendment(s) pending — apply or withdraw before mark_ready"

> **Demo-seed reminder**
> The seeded `demo_booking_globex_001` ships with **no parties** and no POL/POD (route masters aren't seeded). A raw `mark_ready` on it will fail preconditions 5 and 6 — that is expected. Add shipper / consignee / notify parties and a POL/POD on the form first.

Negative checks — each must be refused

#	Action	Expected refusal	✔	Trace
9	`create_from_handover` against a handover that is **already routed**.	Rejected: "handover already routed (fulfillment_target_id=…); cannot create_from_handover (BR-HO-ROUTE-01)".	**Blocked**	BR-HO-ROUTE-01
10	`create_from_handover` against a handover **not** in `accepted` .	Rejected: "cannot create booking from handover in status … — handover must be `accepted` first (AC-12 / BR-BK-02)".	**Blocked**	BR-09 / BR-BK-02
11	`request_provider` before the feasibility check exists / while it is pending.	Rejected: "request_provider blocked — feasibility check missing (BR-BK-03 / QAB-12)" or "feasibility result is 'pending' …".	**Blocked**	BR-BK-03
12	`provider.request.confirm` with an empty `confirmation_ref` .	Rejected: "confirm requires a non-empty `confirmation_ref`".	**Blocked**	BR-BK-CONF-02
13	`provider.request.confirm` a `shipping_line` row missing `vessel_name` / `voyage_number` .	Rejected: "confirm: missing required fields for provider_kind 'shipping_line': […] (BR-BK-CONF-02)".	**Blocked**	BR-BK-CONF-02
14	Edit a locked field (e.g. `total_sell_amount`) on a `confirmed` booking directly.	Rejected: "booking fields […] are immutable in status 'confirmed'; open an amendment to change them (BR-BK-04)".	**Blocked**	BR-BK-04

The immutability in step 14 is exactly why Scenario B's amendment workflow exists — it is the only legal mutation path past `confirmed` , via the `_amendment_apply` bypass.

SECTION 04 · SCENARIO B

04 Feasibility commit & the amendment workflow

Goal: prove the two controlled-change surfaces — the 7-dimension feasibility gate that guards provider requests, and the append-only amendment trail that is the only legal way to change a confirmed booking.

Part 1 — Feasibility (QAB-12)

#	Action	Expected result	✔	Trace
1	On the pending check, `update_finding` for each of the 7 dimensions with `status` ∈ {ok, warn, block}.	Each finding slot written. A bad dimension or status is refused ("dimension must be one of …", "finding status must be one of {'ok','warn','block'}").	**7 findings set.**	QAB-12
2	With all `ok`, run `commit`.	result → **feasible**; `checked_at` stamped. If any dimension unset: "commit blocked — all 7 … must have a finding".	**result=feasible.**	BR-BK-FEAS-03
3	On a fresh check, set one `warn` and `commit` **without** a `summary_note`.	Rejected: "commit with `exception` result requires `summary_note`". Supply the note → result=**exception** (proceed with annotation).	**Note enforced for exception.**	BR-BK-FEAS-03
4	On a fresh check, set one `block`, add a note, and `commit`.	result → **not_feasible**: opens the `booking.feasibility.escalation` approval case (subject = handover) and emits `booking.feasibility.failed` — routing control back to sales-crm.	**Escalation case + event.**	BR-BK-FEAS-04
5	Attempt `update_finding` again after commit.	Rejected: "cannot update_finding once result is …(BR-BK-FEAS-01)" — checks are append-only once terminal.	**Blocked.**	BR-BK-FEAS-01

Part 2 — Amendment (BR-14)

#	Action	Expected result	✔	Trace
6	On a **confirmed** booking, create a `logistics.booking.amendment` with a `change_type` and non-zero `sell_delta_amount`.	Created in **draft**; `requires_commercial_review` computed **true** from the delta; `customer_notification_required` computed from change_type.	**Review flag=true.**	BR-BK-AMD-02
7	Run `amendment.submit`.	With deltas → **pending_approval** + a `booking.amendment.commercial_review` case opened (subject = the amendment row). No deltas/notify → applied immediately.	**status=pending_approval.**	BR-14 / BR-BK-AMD-02
8	On approval, `apply_amendment` fires (or a no-gate submit auto-applies).	`after_values` written onto the booking via the `_amendment_apply` bypass; impacted provider requests flip to **amended**; `booking.amended` emitted; amendment → **applied**.	**Booking mutated via bypass.**	BR-BK-AMD-05
9	Create + `amendment.withdraw` a pending amendment; then try to amend a **draft** booking.	Withdraw → **withdrawn**. Amend on draft rejected: "amendments are only allowed on bookings in status {'confirmed','ready_for_execution'}… (BR-BK-AMD-01)".	**Withdraw ok; draft-amend blocked.**	BR-BK-AMD-01

✔ Scenario B pass criteria

Feasibility gates provider requests (feasible/exception pass, not_feasible escalates); and confirmed-booking changes flow only through the amendment trail with commercial-review approval when money moves.

⚠ Non-amendable changes

`change_type` of `transport_mode` or `source_handover` is refused at create: "… is not amendable — cancel + new booking required (BR-BK-AMD-06)". Those changes mean a new booking, not an amendment.

SECTION 05 · SCENARIO C

05 Cancellation — grace window, fee approval & guards

Goal: prove the grace-aware cancellation path — free inside the window, approval-gated outside it — plus the equipment release cascade and the guards that refuse an illegal cancel. **Start:** a booking in `requested` or `confirmed`.

#	Action	Expected result	✔ Checkpoint	Trace
1	`cancel` a booking **within** `booking.cancellation_grace_hours` (default 24h from `requested_at`) with a `cancellation_reason`.	Synchronous cancel, cost = 0. Status → **cancelled**, open provider requests cascade-cancelled, equipment released, `booking.cancelled` emitted.	**status=cancelled, cost 0.**	BR-15 / BR-BK-CAN-01
2	`cancel` a booking **outside** the grace window.	No immediate flip — a `booking.cancel.fee` approval case opens, `cancellation_pending=true`. Re-calling while pending returns the same case id (idempotent).	**Pending + approval case id.**	BR-15 / BR-BK-CAN-01
3	On the case landing, `complete_cancellation` with `decision=approved`.	Runs the apply path: status → **cancelled**, equipment released, providers cascade-cancelled, `booking.cancelled` emitted.	**Cancelled after approval.**	BR-BK-CAN-01
4	`complete_cancellation` with `decision=rejected`.	Clears `cancellation_pending`; booking stays live in its prior status.	**Pending cleared, not cancelled.**	BR-BK-CAN-01
5	`allocate_equipment_usage` then `release_equipment_usage` on the booking.	Allocation rows created (idempotent — re-run returns existing); release marks all `released_at` and cascades in-house usage cancel to equipment-operations. Degrades cleanly if that module isn't loaded.	**Allocate idempotent; release audited.**	Integration

Negative checks — each must be refused

#	Action	Expected refusal	Trace
6	`cancel` with an empty `cancellation_reason`.	"cancel requires a non-empty cancellation_reason (BR-BK-CAN-05)".	BR-BK-CAN-05
7	`cancel` a booking in `ready_for_execution`.	"cannot cancel a booking in `ready_for_execution` — cancellation must go via the downstream shipment cancellation flow (BR-BK-CAN-02)".	BR-BK-CAN-02
8	`complete_cancellation` with a decision other than approved/rejected.	"complete_cancellation requires payload.decision in {'approved','rejected'}; got …".	BR-BK-CAN-01

Config toggles worth flipping during the run

Config key (`ir.config`)	Default	Effect to verify
`booking.cancellation_grace_hours`	24	Lower it to force step 1 outside grace → the fee-approval path.
`booking.feasibility_required`	true	false → `request_provider` skips the feasibility gate (§03 step 11).
`booking.provider.require_role_match`	true	Gates whether a provider partner must hold the role matching its kind.
`booking.confirm.require_document`	false	true → `confirm` demands a `confirmation_document_id`.

✔ Full-module pass criteria

All three scenarios green: Y-fork Path B carry-forward + provider confirm + readiness (A), feasibility commit + amendment with commercial review (B), and grace-vs-approval cancellation with equipment release (C) — with every negative guard in §03–§05 confirmed to refuse.

Traceability: `BR-*` / `QAB-*` IDs map to `roadmap/logistics/booking/brs.md`; the finer `BR-BK-*` codes are the in-code requirement anchors. The `Booking.ReadyForExecution` event is the boundary consumed by the Shipments module — verified in that module's own test guide.